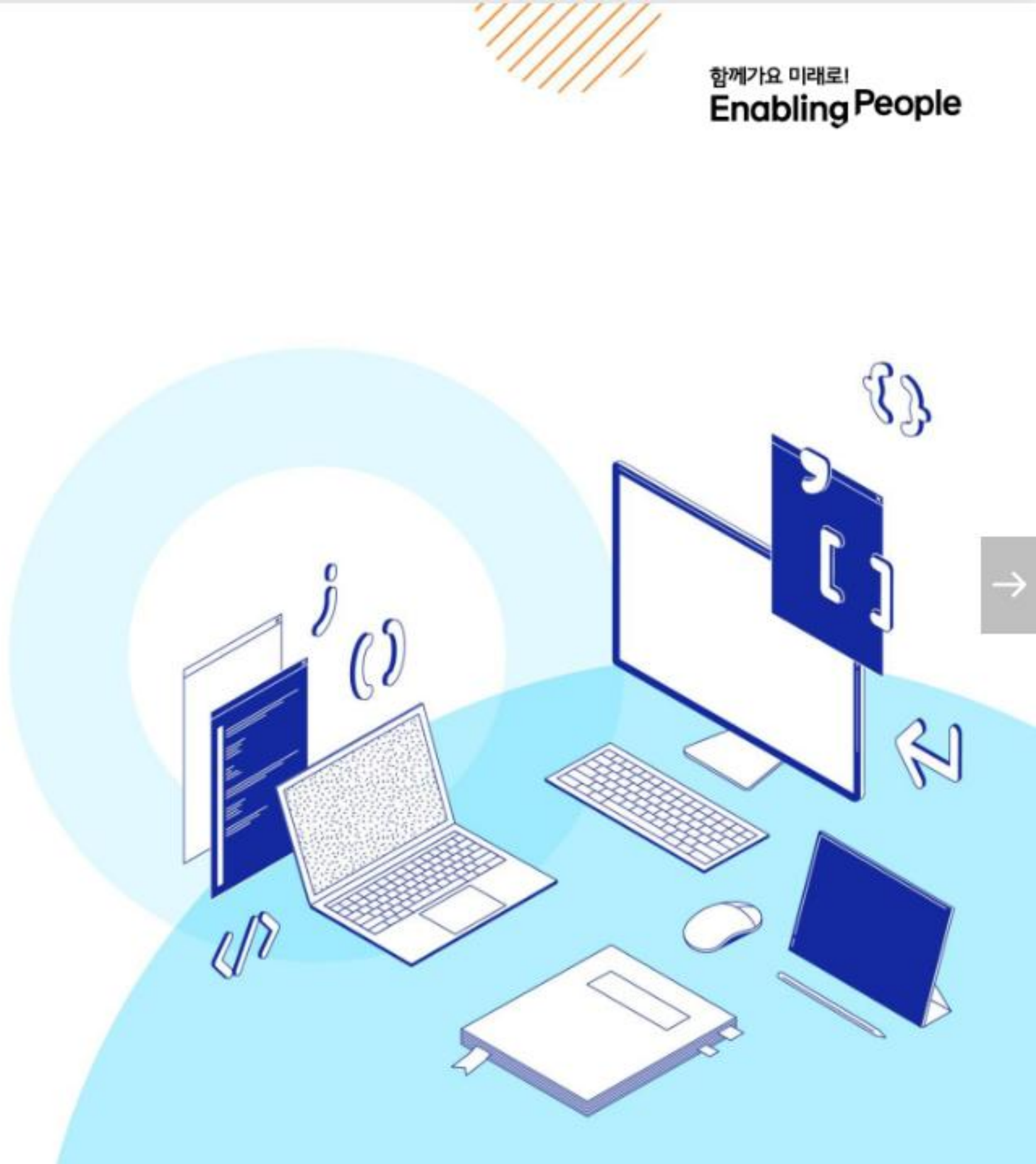


SSAFY

DB

SQL 01

Python



목 차

Database

Relational Database

- RDBMS

SQL

- SQL Statements

Querying data

- SELECT

Sorting data

- ORDER BY

목 차

Filtering data

- DISTINCT
- WHERE
- Operators
- LIMIT

Grouping data

- GROUP BY
- HAVING

참고

- Query
- NULL 비교
- SQL 표준

SQL 활용 방법

- "고객 정보는 어떻게 저장되고 있을까?"

1. 수많은 고객의 이름, 주소, 주문 내역을 모두 엑셀로 관리할 수 있을까요?
2. 같은 이름을 가진 고객이 여러 명이라면 어떻게 구별해야 할까요?
3. 고객이 한 명이라도 주문을 여러 번 했다면, 그 관계는 어떻게 저장되어야 할까요?

- 데이터를 구조적으로 저장하는 관계형 데이터베이스의 방식을 이해할 수 있어요.
- 고객 정보에서 원하는 조건의 데이터만 SQL로 정확하게 조회하는 방법을 배울 수 있어요.
- 테이블 간 관계를 맺는 방식과 이를 활용하는 기본 쿼리 작성법을 익힐 수 있어요

학습 목표

- ✓ 데이터베이스의 개념과 등장 배경을 이해한다.
- ✓ 관계형 데이터베이스(RDB)의 구조와 테이블 간의 관계를 설명할 수 있다.
- ✓ SQL의 기본 구조와 문법을 이해한다.
- ✓ SELECT, FROM, WHERE, ORDER BY 문을 활용해 데이터를 조회할 수 있다.
- ✓ DISTINCT, LIMIT, LIKE 등의 키워드를 사용하여 원하는 조건으로 데이터를 필터링할 수 있다.
- ✓ GROUP BY와 HAVING을 활용한 집계 데이터를 조회할 수 있다.
- ✓ SQL 실행 순서를 이해하고 각 절의 역할을 구분할 수 있다

Database

Database란?

Database란?

데이터베이스

Database

체계적으로 정리된 데이터의 모음

책장을 떠올려 봅시다.

- 우리가 책을 아무 데나 쌓아두면, 나중에 필요한 책을 찾기 어렵죠.
- 그래서 주제별, 작가별로 분류해서 책장을 정리해 둡니다.

이처럼 컴퓨터도 수많은 정보를 빠르게 찾고, 효율적으로 관리하려면, 잘 정리된 책장처럼 데이터를 구조적으로 저장해야 합니다. 이 역할을 해주는 것이 바로 데이터베이스입니다.

Data란?

데이터

저장이나 처리를 위해 변환된 정보

우리가 표현한 정보들을 컴퓨터가 다룰 수 있도록 구조화한 것이 바로 데이터입니다.
데이터는 분석, 계산, 저장을 위해 정보를 구체적인 형태로 정리한 것이라 할 수 있습니다.

증가하는 데이터 사용량

- 배달의 민족 - 2022년 한 해 동안 11억 1100만 건 이상의 주문을 처리
- 유튜브 - 1분마다 500시간 이상의 새로운 동영상이 업로드
- 넷플릭스 - 구독자 3억 명 돌파(2024), 2025년 상반기 동안 전 세계 시청 시간 약 950억 시간
- IBM - 전세계 모든 데이터의 약 90%는 2015년 이후 생산된 것이라 발표

데이터 센터의 성장

- 네이버 - 제2데이터센터에 6,500억 원을 투자 (2020)
- 카카오 - 제1데이터센터와 제2데이터센터에 1.5조 원을 투자 (2022)

TIP

전 세계 데이터 센터 시장은 2024년부터 2029년까지 연평균 11.13% 성장하여, 2029년에는 약 4,386억 8천만 달러 규모에 이를 것으로 예상됨

- Mordor Intelligence, "Data Center Market Size & Share Analysis"

데이터를 저장하고 잘 관리하여 활용할 수 있는 기술이 중요해짐



<그림1_DB_SQL1_데이터 관리의 중요>

➤우리가 알고 있는 데이터 저장 방식은 어떤 것이 있을까?

1. 파일(File) 이용

- 어디에서나 쉽게 사용 가능
- 데이터를 구조적으로 관리하기 어려움

개인정보.txt

이름: 김한웅
나이: 56
사는 곳: 서울

이름: 유미선
나이: 21
사는 곳: 강원

이름: 박지수
나이: 36
사는 곳: 경기

〈그림2_DB_SQL 1_파일로 데이터 관리하기〉

2. 스프레드 시트(Spreadsheet) 이용

- 테이블의 열과 행을 사용해 데이터를 구조적으로 관리 가능

id	name	age	city
1	김한웅	56	서울
2	유미선	21	강원
3	박지수	36	경기

〈그림3_DB_SQL 1_스프레드 시트로 데이터 관리하기〉

스프레드 시트의 한계

- 크기
 - 일반적으로 약 100만 행까지만 저장가능
- 보안
 - 단순히 파일이나 링크 소유 여부에 따른 단순한 접근 권한 기능 제공
- 정확성
 - 만약 공식적으로 “강원”의 지명이 “강언”으로 바뀌었다고 가정한다면?
 - 이 변경으로 인해 테이블 모든 위치에서 해당 값을 업데이트 해야 함
 - 만약 데이터가 여러 시트에 분산되어 있다면 변경에 누락이 생기거나 추가 문제가 발생할 수 있음

데이터베이스 역할

데이터베이스 역할

Create Read
Update Delete

Relational Database

Relational Database란?

데이터베이스 역할

데이터베이스 역할

데이터를 저장하고 조작

구조적 저장

CRUD

Relational Database란?

관계형 데이터베이스

Relational Database

데이터 간에 **관계**가 있는
데이터 항목들의 모음

병원 접수창구를 떠올려 봅시다.

- 환자 정보는 한쪽 종이에, 진료 기록은 다른 종이에 적혀 있어요.
- 환자 이름이 같은 사람이 여럿이라면, 진료 기록만 보고는 누가 누구인지 헷갈릴 수 있겠죠?

그래서 병원은 **환자마다 고유한 번호(예: 환자번호)**를 부여하고, 진료 기록에는 그 번호를 함께 적습니다. 이렇게 하면 환자 정보와 진료 기록을 정확히 연결할 수 있습니다.

관계형 데이터베이스

- 테이블, 행, 열의 정보를 구조화하는 방식
- 서로 관련된 데이터 포인터를 저장하고 이에 대한 액세스를 제공

주문 테이블

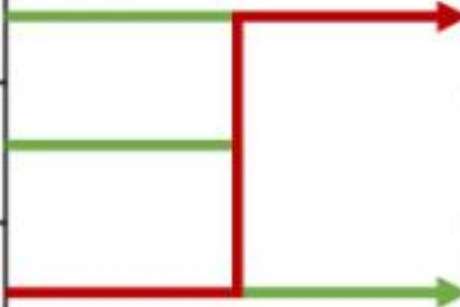
id	주문일	주문 상태	고객 ID
1	2002/01/01	배송완료	3
2	2002/03/04	상품 준비중	3
3	2002/02/03	배송중	1

<표1_DB_SQL 1_주문 테이블>

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

<표2_DB_SQL 1_고객 테이블>



데이터베이스 역할

관계

여러 테이블 간의 (논리적) 연결

데이터를 각각의 테이블에 나눠 저장하되,
공통된 키 값을 통해 서로 연결하여 필요할 때 함께 조회하거나 조작할 수 있도록 합니다.

관계로 할 수 있는 것

- 이 관계로 인해 두 테이블을 사용하여 데이터를 다양한 형식으로 조회할 수 있음
 - 특정 날짜에 구매한 모든 고객 조회
 - 지난 달에 배송일이 지연된 고객 조회 등

주문 테이블

id	주문일	주문 상태	고객 ID
1	2002/01/01	배송완료	3
2	2002/03/04	상품 준비중	3
3	2002/02/03	배송중	1

〈표1_DB_SQL 1_주문 테이블〉

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

〈표2_DB_SQL 1_고객 테이블〉

관계형 데이터베이스 예시 (1/4)

- 다음과 같이 고객 데이터가 테이블에 저장되어 있다고 가정
- 고객 데이터 간 비교를 위해서는 어떤 값을 활용해야 할까?
 - 이름? 주소? 만약 동명이인이나 같은 주소지가 있다면?

고객 테이블

이름	청구지	주소지
김한웅	서울	강원
유미선	강원	경기
박지수	경기	서울

<표2_DB_SQL 1_고객 테이블>

관계형 데이터베이스 예시 (2/4)

- 다음과 같이 고객 데이터가 테이블에 저장되어 있다고 가정
- 고객 데이터 간 비교를 위해서는 어떤 값을 활용해야 할까?
 - 각 데이터에 고유한 식별 값을 부여하기 (기본 키, Primary Key)

ex) 주민등록번호 고객 테이블

Id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

〈표2_DB_SQL1_고객 테이블〉

관계형 데이터베이스 예시 (3/4)

- 다음과 같이 각 고객이 주문한 주문데이터가 테이블에 저장되어 있다고 가정
- 누가 어떤 주문을 했는지 어떻게 식별할 수 있을까?
 - 고객 이름? 마찬가지로 동명이인이 있다면?

주문 테이블

id	주문일	주문 상태
1	2002/01/01	배송완료
2	2002/03/04	상품 준비중
3	2002/02/03	배송중

<표1_DB_SQL 1_주문 테이블>

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

<표2_DB_SQL 1_고객 테이블>

관계형 데이터베이스 예시 (4/4)

- 다음과 같이 각 고객이 주문한 주문데이터가 테이블에 저장되어 있다고 가정
- 누가 어떤 주문을 했는지 어떻게 식별할 수 있을까?
 - 주문 정보에 고객의 고유한 식별 값을 저장하기 (외래 키, Foreign Key)

주문 테이블

id	주문일	주문 상태	고객 ID
1	2002/01/01	배송완료	3
2	2002/03/04	상품 준비중	2
3	2002/02/03	배송중	1

<표1_DB_SQL 1_주문 테이블>

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

<표2_DB_SQL 1_고객 테이블>

관계형 데이터베이스 관련 키워드 (1/6)

1. Table (*aka* Relation)

- 데이터를 기록하는 곳

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

<표2_DB_SQL 1_고객 테이블>

관계형 데이터베이스 관련 키워드 (2/6)

1. Table (*aka* Relation)

- 데이터를 기록하는 곳

2. Field (*aka* Column, Attribute)

- 각 필드에는 고유한 데이터 형식(타입)이 지정됨

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

<표2_DB_SQL-1_고객 테이블>

관계형 데이터베이스 관련 키워드 (3/6)

1. Table (*aka* Relation)

- 데이터를 기록하는 곳

2. Field (*aka* Column, Attribute)

- 각 필드에는 고유한 데이터 형식(타입)이 지정됨

3. Record (*aka* Row, Tuple)

- 각 레코드에는 구체적인 데이터 값이 저장됨

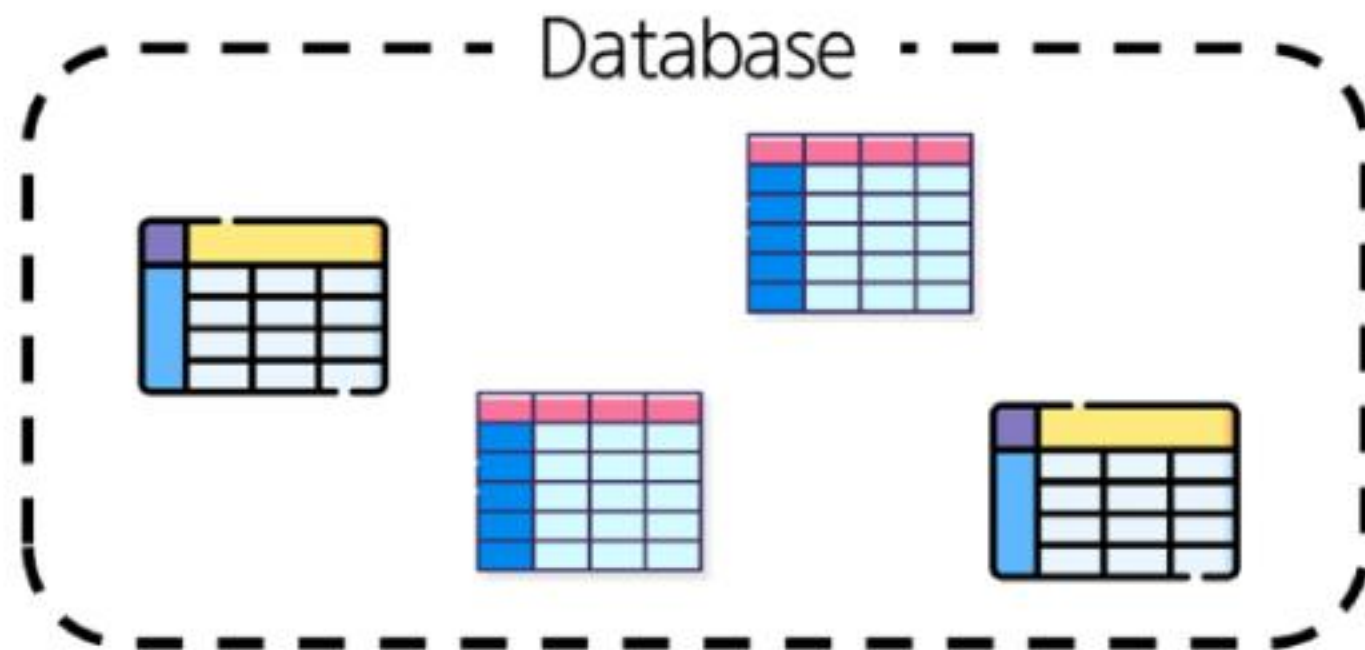
고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

<표2_DB_SQL 1_고객 테이블>

관계형 데이터베이스 관련 키워드 (4/6)

1. Table (*aka* Relation)
 - 데이터를 기록하는 곳
2. Field (*aka* Column, Attribute)
 - 각 필드에는 고유한 데이터 형식(타입)이 지정됨
3. Record (*aka* Row, Tuple)
 - 각 레코드에는 구체적인 데이터 값이 저장됨
4. Database (*aka* Schema)
 - 테이블의 집합



<그림4_DB_SQL 1_데이터베이스 예시>

관계형 데이터베이스 관련 키워드 (5/6)

5. Primary Key (기본 키, PK)

- 각 레코드의 고유한 값
- 관계형 데이터베이스에서 레코드의 식별자로 활용

주문 테이블

id	주문일	주문 상태	고객 ID
1	2002/01/01	배송완료	3
2	2002/03/04	상품 준비중	2
3	2002/02/03	배송중	1

<표1_DB_SQL 1_주문 테이블>

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

<표2_DB_SQL 1_고객 테이블>

관계형 데이터베이스 관련 키워드 (6/6)

6. Foreign Key (외래 키, FK)

- 테이블의 필드 중 다른 테이블의 레코드를 식별할 수 있는 키
- 다른 테이블의 기본 키를 참조
- 각 레코드에서 서로 다른 테이블 간의 관계를 만드는 데 사용

주문 테이블

id	주문일	주문 상태	고객 ID
1	2002/01/01	배송완료	3
2	2002/03/04	상품 준비중	2
3	2002/02/03	배송중	1

〈표1_DB_SQL 1_주문 테이블〉

고객 테이블

id	이름	청구지	주소지
1	김한웅	서울	강원
2	유미선	강원	경기
3	박지수	경기	서울

〈표2_DB_SQL 1_고객 테이블〉

RDBMS

DBMS란?

DBMS

Database Management System

데이터베이스를 관리하는
소프트웨어 프로그램

DBMS는 컴퓨터 안에서 '정리된 서류함 역할'을 합니다.

- 데이터를 일정한 규칙으로 저장하고,
- 필요할 때 쉽게 꺼내거나 바꿀 수 있도록 도와주는 도구예요.

DBMS

- 데이터 저장 및 관리를 용이하게 하는 시스템
- 데이터베이스와 사용자 간의 인터페이스 역할
- 사용자가 데이터 구성, 업데이트, 모니터링, 백업, 복구 등을 할 수 있도록 도움



DBMS

데이터베이스를 관리하는
소프트웨어 프로그램

<개념1_DB_SQL 1_DBMS 개념>

DBMS란?

RDBMS

Relational Database Management System

관계형 데이터베이스 관리
소프트웨어 프로그램

도서관의 대출 시스템을 떠올려 봅시다.

- 책은 도서 목록, 사람은 회원 명단, 대출 내역은 대출 기록이라는 표로 관리된다고 할게요.
- 이 세 표는 각각 따로 있지만, 회원 번호와 책 번호를 통해 서로 연결돼 있죠.

RDBMS는 이런 식으로 데이터를 여러 테이블에 나누어 저장하되,

공통된 키를 통해 서로 '관계'를 맺고 함께 사용할 수 있게 해주는 시스템입니다.

RDBMS 서비스 종류

- SQLite
- MySQL
- PostgreSQL
- Oracle Database
- ...



RDBMS

관계형 데이터베이스 관리
소프트웨어 프로그램

<개념2_DB_SQL 1_RDBMS 개념>

TIP

프로젝트의 목적과 환경에 따라 RDBMS를 선택하세요.
RDBMS는 모두 SQL을 기반으로 동작하지만, 세부 기능이나 사용 방식은 조금씩 다르니 프로젝트에 맞는
도구를 고르는 것이 중요합니다.

SQLite

- 경량의 오픈 소스 데이터베이스 관리 시스템
- 설치 없이 가볍게 실행 가능해 모바일 앱이나 소규모 프로그램에 적합
- 컴퓨터나 모바일 기기에 내장되어 간단하고 효율적인 데이터 저장 및 관리를 제공



〈그림5_DB_SQL 1_SQLite 로고〉

데이터베이스 정리

- Table은 데이터가 기록되는 곳
- Table에는 행에서 고유하게 식별 가능한 기본 키라는 속성이 있으며,
외래 키를 사용하여 각 행에서 서로 다른 테이블 간의 관계를 만들 수 있음
- 데이터는 기본 키 또는 외래 키를 통해 결합(join)될 수 있는 여러 테이블에 걸쳐 구조화 됨



데이터베이스
체계적으로 정리된 데이터의
모음

<개념3_DB_SQL 1_데이터베이스 정의>

SQL

SQL이란?

SQL이란?

SQL

Structure Query Language

테이블의 형태로 **구조화** 된 관계형
데이터베이스에게 요청을 **질의(요청)**

SQL은 도서관 사서를 위한 요청서입니다.

SQL은 데이터베이스라는 거대한 도서관에서, '사서' 역할을 하는 데이터베이스 관리 시스템(DBMS)에게 내가 원하는 데이터를 찾아달라고 요청하는 **표준화된 요청서 양식**과 같습니다.

"2000년 이후에 출판된 과학 소설 중에서, 책 제목과 저자 정보를 최신순으로 정렬해서 찾아주세요."

SQL은 사람이 사용하는 모호한 말이 아닌 **명확하게 약속된 형식**에 따라 데이터베이스에 정확한 요청을 전달하는 언어입니다.

SQL Syntax

```
SELECT column_name FROM table_name;
```

1. SQL 키워드는 대소문자를 구분하지 않음

- 하지만 대문자로 작성하는 것을 권장 (명시적 구분)

2. 각 SQL Statements의 끝에는 세미콜론(';')이 필요

- 세미콜론은 각 SQL Statements를 구분하는 방법 (명령어의 마침표)



SQL

테이블의 형태로 구조화 된 관계형
데이터베이스에게 요청을 질의(요청)

<개념4_DB_SQL1_SQL 정의>

SQL Statements

SQL Statements

- SQL을 구성하는 가장 기본적인 코드 블록

수행 목적에 따른 SQL Statements 4가지 유형

1. DDL - 데이터 정의
2. DQL - 데이터 검색
3. DML - 데이터 조작
4. DCL - 데이터 제어

SQL Statements 예시

```
SELECT column_name FROM table_name;
```

- 해당 예시 코드는 SELECT Statement라 부름
- Statement는 SELECT, FROM 2개의 keyword로 구성

수행 목적에 따른 SQL Statements 4가지 유형

※ 단순히 SQL 문법을 암기하고 실행만 하는 것이 아닌 관계형 데이터베이스를 잘 이해하고 다루는 방법을 학습

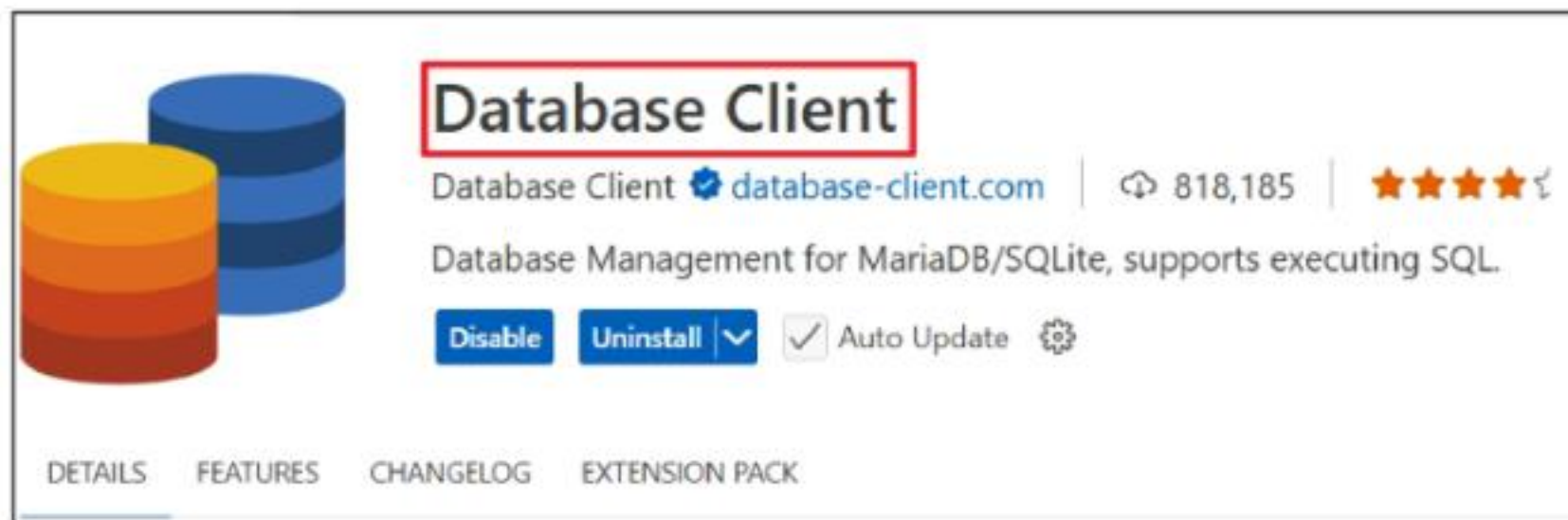
유형	역할	SQL 키워드
DDL (Data Definition Language)	데이터의 기본 구조 및 형식 변경	CREATE DROP ALTER
DQL (Data Query Language)	데이터 검색	SELECT
DML (Data Manipulation Language)	데이터 조작 (추가, 수정, 삭제)	INSERT UPDATE DELETE
DCL (Data Control Language)	데이터 및 작업에 대한 사용자 권한 제어	COMMIT ROLLBACK GRANT REVOKE

Querying data

사전 준비

Database 연결

- VSCode 확장 툴 Database Client 사용
- 사전에 제공한 **sample.sqlite3** DB 연결
- 연결 방법은 공용 노션 문서 참고



<그림6_DB_SQL 1_Database Client 확장 툴>



<그림7_DB_SQL 1_공용 노션 예시>

DQL

SQL Statements 유형

유형	역할	SQL 키워드
DDL (Data Definition Language)	데이터의 기본 구조 및 형식 변경	CREATE DROP ALTER
DQL (Data Query Language)	데이터 검색	SELECT
DML (Data Manipulation Language)	데이터 조작 (추가, 수정, 삭제)	INSERT UPDATE DELETE
DCL (Data Control Language)	데이터 및 작업에 대한 사용자 권한 제어	COMMIT ROLLBACK GRANT REVOKE

〈표4_DB_SQL1_SQL문 4가지 유형 중 DQL〉

SELECT

SELECT syntax

```
SELECT  
    select_list  
FROM  
    table_name;
```

- SELECT 키워드 이후 데이터를 선택하려는 필드를 하나 이상 지정
- FROM 키워드 이후 데이터를 선택하려는 테이블의 이름을 지정



SELECT statement
테이블에서 데이터를 조회

〈개념5_DB_SQL 1_SELECT 문 정의〉

SELECT 활용 1

- 테이블 employees에서 LastName 필드의 모든 데이터를 조회

Q	* LastName NVARCHAR(20)
>	Adams
>	Edwards
>	Peacock
>	Park
>	Johnson
>	Mitchell
>	King
>	Callahan

<그림8_DB_SQL 1_SELECT 활용 1 실행 결과>

SELECT 활용 1

- 테이블 employees에서 LastName 필드의 모든 데이터를 조회

```
SELECT  
    LastName  
FROM  
    employees;
```

Q * LastName NVARCHAR(20)  

>	Adams
>	Edwards
>	Peacock
>	Park
>	Johnson
>	Mitchell
>	King
>	Callahan

<그림8_DB_SQL 1_SELECT 활용 1 실행 결과>

SELECT 활용 2

- 테이블 employees에서 LastName, FirstName 필드의 모든 데이터를 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(20)
>	Adams	Andrew
>	Edwards	Nancy
>	Peacock	Jane
>	Park	Margaret
>	Johnson	Steve
>	Mitchell	Michael
>	King	Robert
>	Callahan	Laura

〈그림9_DB_SQL 1_SELECT 활용 2 실행 결과〉

SELECT 활용 2

- 테이블 employees에서 LastName, FirstName 필드의 모든 데이터를 조회

```
SELECT  
    LastName  
FROM  
    employees;
```

	* LastName NVARCHAR(20)	* FirstName NVARCHAR(20)
>	Adams	Andrew
>	Edwards	Nancy
>	Peacock	Jane
>	Park	Margaret
>	Johnson	Steve
>	Mitchell	Michael
>	King	Robert
>	Callahan	Laura

SELECT 활용 3

- 테이블 employees에서 모든 필드 데이터를 조회

EmployeeId INTEGER	LastName NVARCHAR(25)	FirstName NVARCHAR(25)	Title NVARCHAR(30)	ReportsTo INTEGER	BirthDate DATETIME	HireDate DATETIME	Address NVARCHAR(70)	City NVARCHAR(45)	State NVARCHAR(45)	Country NVARCHAR(45)	PostalCode NVARCHAR(15)	Phone NVARCHAR(24)	Fax NVARCHAR(24)	Email NVARCHAR(60)
1	Adams	Andrew	General Manager	(NULL)	1962-02-18 00:00:00	2002-08-14 00:00:00	11120 Jasper Ave NW	Edmonton	AB	Canada	T5K 2N1	+1 (780) 428-9482	+1 (780) 428-3457	andrew@chinookcorp.com
2	Edwards	Nancy	Sales Manager	1	1958-12-08 00:00:00	2002-05-01 00:00:00	825 8 Ave SW	Calgary	AB	Canada	T2P 2T3	+1 (403) 262-3443	+1 (403) 262-3322	nancy@chinookcorp.com
3	Pearcock	Jane	Sales Support Agent	2	1973-08-29 00:00:00	2002-04-01 00:00:00	1111 6 Ave SW	Calgary	AB	Canada	T2P 5M5	+1 (403) 262-3443	+1 (403) 262-6712	jane@chinookcorp.com
4	Park	Margaret	Sales Support Agent	2	1947-09-19 00:00:00	2003-05-03 00:00:00	683 10 Street SW	Calgary	AB	Canada	T2P 5G3	+1 (403) 263-4423	+1 (403) 263-4289	margaret@chinookcorp.com
5	Johnson	Steve	Sales Support Agent	2	1965-03-03 00:00:00	2003-10-17 00:00:00	77278 41 Ave	Calgary	AB	Canada	T3B 1Y7	1 (780) 836-9987	1 (780) 836-9543	steve@chinookcorp.com
6	Mitchell	Michael	IT Manager	1	1973-07-01 00:00:00	2003-10-17 00:00:00	5827 Bowness Road NW	Calgary	AB	Canada	T3B 0C5	+1 (403) 246-9887	+1 (403) 246-9899	michael@chinookcorp.com
7	King	Robert	IT Staff	6	1970-05-29 00:00:00	2004-01-02 00:00:00	590 Columbia Boulevard W	Lethbridge	AB	Canada	T1K 5N8	+1 (403) 456-9986	+1 (403) 456-8485	robert@chinookcorp.com
8	Callahan	Laura	IT Staff	6	1968-01-09 00:00:00	2004-03-04 00:00:00	923 7 ST NW	Lethbridge	AB	Canada	T1H 1Y8	+1 (403) 467-3351	+1 (403) 467-8772	laura@chinookcorp.com

<그림10_DB_SQL 1_SELECT 활용 3 실행 결과>

SELECT 활용 3

- 테이블 employees에서 모든 필드 데이터를 조회

```
SELECT * FROM employees;
```

EmployeeID INTEGER	LastName NVARCHAR(20)	FirstName NVARCHAR(20)	Title NVARCHAR(30)	ReportsTo INTEGER	BirthDate DATETIME	HireDate DATETIME	Address NVARCHAR(70)	City NVARCHAR(40)	State NVARCHAR(40)	Country NVARCHAR(40)	PostalCode NVARCHAR(10)	Phone NVARCHAR(24)	Fax NVARCHAR(24)	Email NVARCHAR(60)
1	Adams	Andrew	General Manager	(NULL)	1962-02-18 00:00:00	2002-08-14 00:00:00	11120 Jasper Ave NW	Edmonton	AB	Canada	T5K 2N1	+1 (780) 428-9482	+1 (780) 428-3457	andrew@chinookcorp.com
2	Edwards	Nancy	Sales Manager	1	1958-12-08 00:00:00	2002-05-01 00:00:00	825 8 Ave SW	Calgary	AB	Canada	T2P 2T3	+1 (403) 262-3443	+1 (403) 262-3322	nancy@chinookcorp.com
3	Peacock	Jane	Sales Support Agent	2	1973-08-29 00:00:00	2002-04-01 00:00:00	1111 6 Ave SW	Calgary	AB	Canada	T2P 5M5	+1 (403) 262-3443	+1 (403) 262-6712	jane@chinookcorp.com
4	Park	Margaret	Sales Support Agent	2	1947-09-19 00:00:00	2003-05-03 00:00:00	683 10 Street SW	Calgary	AB	Canada	T2P 5G3	+1 (403) 263-4423	+1 (403) 263-4289	margaret@chinookcorp.com
5	Johnson	Steve	Sales Support Agent	2	1965-03-03 00:00:00	2003-10-17 00:00:00	77278 41 Ave	Calgary	AB	Canada	T3B 1Y7	1 (780) 836-9987	1 (780) 836-9543	steve@chinookcorp.com
6	Mitchell	Michael	IT Manager	1	1973-07-01 00:00:00	2003-10-17 00:00:00	5827 Bowness Road NW	Calgary	AB	Canada	T3B 0C5	+1 (403) 246-9887	+1 (403) 246-9899	michael@chinookcorp.com
7	King	Robert	IT Staff	6	1970-05-29 00:00:00	2004-01-02 00:00:00	590 Columbia Boulevard W	Lethbridge	AB	Canada	T1K 5N8	+1 (403) 456-9986	+1 (403) 456-8485	robert@chinookcorp.com
8	Callahan	Laura	IT Staff	6	1968-01-09 00:00:00	2004-03-04 00:00:00	923 7 ST NW	Lethbridge	AB	Canada	T1H 1Y8	+1 (403) 467-3351	+1 (403) 467-8772	laura@chinookcorp.com

<그림10_DB_SQL_1_SELECT 활용 3 실행 결과>

SELECT 활용 4

- 테이블 employees에서 FirstName 필드의 모든 데이터를 조회
 - (단, 조회 시 FirstName이 아닌 '이름' 으로 출력 될 수 있도록 변경)

이름
Andrew
Nancy
Jane
Margaret
Steve
Michael
Robert
Laura

<그림11_DB_SQL 1_SELECT 활용 4 실행 결과>

SELECT 활용 4

- 테이블 employees에서 FirstName 필드의 모든 데이터를 조회
 - (단, 조회 시 FirstName이 아닌 '이름' 으로 출력 될 수 있도록 변경)

```
SELECT  
  FirstName AS '이름'  
FROM  
  employees;
```

이름

> Andrew
> Nancy
> Jane
> Margaret
> Steve
> Michael
> Robert
> Laura

<그림11_DB_SQL 1_SELECT 활용 4 실행 결과>

SELECT 활용 5

- 테이블 tracks에서 Name, Milliseconds 필드의 모든 데이터 조회
 - (단, Milliseconds 필드는 60000으로 나눠 분 단위 값으로 출력)

Q	* Name NVARCHAR(200)	재생 시간(분)
>	For Those About To Rock (V	5
>	Balls to the Wall	5
>	Fast As a Shark	3
>	Restless and Wild	4
>	Princess of the Dawn	6
>	Put The Finger On You	3
>	Let's Get It Up	3
>	Inject The Venom	3
>	Snowballed	3
>	Evil Walks	4
>	C.O.D.	3
>	Breaking The Rules	4

<그림12_DB_SQL 1_SELECT 활용 5 실행 결과>

SELECT 활용 5

- 테이블 tracks에서 Name, Milliseconds 필드의 모든 데이터 조회
 - (단, Milliseconds 필드는 60000으로 나눠 분 단위 값으로 출력)

```
SELECT
  Name,
  Milliseconds / 60000 AS '재생 시간(분)'
FROM
  tracks;
```

Name	재생 시간(분)
For Those About To Rock (V	5
Balls to the Wall	5
Fast As a Shark	3
Restless and Wild	4
Princess of the Dawn	6
Put The Finger On You	3
Let's Get It Up	3
Inject The Venom	3
Snowballed	3
Evil Walks	4
C.O.D.	3
Breaking The Rules	4

〈그림12_DB_SQL 1_SELECT 활용 5 실행 결과〉

SELECT 정리

- 테이블의 데이터를 조회 및 반환
- ‘*’ (asterisk)를 사용하여 모든 필드 선택



SELECT statement
테이블에서 데이터를 조회

〈개념5_DB_SQL 1_SELECT 문 정의〉

Sorting data

ORDER BY

ORDER BY syntax

```
SELECT
    select_list
FROM
    table_name
ORDER BY
    column1 [ASC|DESC],
    column2 [ASC|DESC],
    ...;
```



ORDER BY statement
조회 결과의 레코드를 정렬

<개념6_DB_SQL 1_ORDER BY 문 정의>

- FROM clause 뒤에 위치
- 하나 이상의 컬럼을 기준으로 결과를 오름차순(ASC, 기본 값), 내림차순(DESC)으로 정렬

ORDER BY 활용 1

- 테이블 employees에서 FirstName 필드의 모든 데이터를 오름차순으로 조회

Q	* FirstName	
	NVARCHAR(20)	
>	Andrew	
>	Jane	
>	Laura	
>	Margaret	
>	Michael	
>	Nancy	
>	Robert	
>	Steve	

<그림13_DB_SQL 1_ORDER BY 활용 1 실행 결과>

ORDER BY 활용 1

- 테이블 employees에서 FirstName 필드의 모든 데이터를 오름차순으로 조회

```
SELECT  
    FirstName  
FROM  
    employees  
ORDER BY  
    FirstName;
```

	* FirstName NVARCHAR(20)
>	Andrew
>	Jane
>	Laura
>	Margaret
>	Michael
>	Nancy
>	Robert
>	Steve

ORDER BY 활용 2

- 테이블 employees에서 FirstName 필드의 모든 데이터를 내림차순으로 조회

Q	* FirstName	
	NVARCHAR(20)	
>	Steve	
>	Robert	
>	Nancy	
>	Michael	
>	Margaret	
>	Laura	
>	Jane	
>	Andrew	

ORDER BY 활용 2

- 테이블 employees에서 FirstName 필드의 모든 데이터를 내림차순으로 조회

```
SELECT  
    FirstName  
FROM  
    employees  
ORDER BY  
    FirstName DESC;
```

	* FirstName NVARCHAR(20)
>	Steve
>	Robert
>	Nancy
>	Michael
>	Margaret
>	Laura
>	Jane
>	Andrew

ORDER BY 활용 3

- 테이블 customers에서 Country 필드를 기준으로 내림차순 정렬한 다음 City 필드 기준으로 오름차순 정렬하여 조회

	Country NVARCHAR(4C) ▾ ▴	City NVARCHAR(4C) ▾ ▴
>	United Kingdom	Edinburgh
>	United Kingdom	London
>	United Kingdom	London
>	USA	Boston
>	USA	Chicago
>	USA	Cupertino
>	USA	Fort Worth
>	USA	Madison
>	USA	Mountain View
>	USA	Mountain View

<그림15_DB_SQL 1_ORDER BY 활용 3 실행 결과>

ORDER BY 활용 3

- 테이블 customers에서 Country 필드를 기준으로 내림차순 정렬한 다음
City 필드 기준으로 오름차순 정렬하여 조회

```
SELECT  
    Country, City  
FROM  
    customers  
ORDER BY  
    Country DESC, City;
```

	Country NVARCHAR(4C)	City NVARCHAR(4C)
>	United Kingdom	Edinburgh
>	United Kingdom	London
>	United Kingdom	London
>	USA	Boston
>	USA	Chicago
>	USA	Cupertino
>	USA	Fort Worth
>	USA	Madison
>	USA	Mountain View
>	USA	Mountain View

〈그림15_DB_SQL 1_ORDER BY 활용 3 실행 결과〉

ORDER BY 활용 4

- 테이블 tracks에서 Milliseconds 필드를 기준으로 내림차순 정렬한 다음 Name, Milliseconds 필드의 모든 데이터를 조회
 - (단, Milliseconds 필드는 60,000으로 나눠 분 단위 값으로 출력)

Q	* Name NVARCHAR(200)	재생 시간(분)
>	Occupation / Precipice	88
>	Through a Looking Glass	84
>	Greetings from Earth, Pt. 1	49
>	The Man With Nine Lives	49
>	Battlestar Galactica, Pt. 2	49
>	Battlestar Galactica, Pt. 1	49

<그림16_DB_SQL 1_ORDER BY 활용 4 실행 결과>

ORDER BY 활용 4

- 테이블 tracks에서 Milliseconds 필드를 기준으로 내림차순 정렬한 다음
Name, Milliseconds 필드의 모든 데이터를 조회
 - (단, Milliseconds 필드는 60,000으로 나눠 분 단위 값으로 출력)

```
SELECT
  Name,
  Milliseconds / 60000 AS '재생 시간(분)'
FROM
  tracks
ORDER BY
  Milliseconds DESC;
```

	* Name NVARCHAR(200)	재생 시간(분)
>	Occupation / Precipice	88
>	Through a Looking Glass	84
>	Greetings from Earth, Pt. 1	49
>	The Man With Nine Lives	49
>	Battlestar Galactica, Pt. 2	49
>	Battlestar Galactica, Pt. 1	49

정렬에서의 NULL

- NULL 값이 존재할 경우 오름차순 정렬 시 결과에 NULL이 먼저 출력

```
SELECT  
  ReportsTo  
FROM  
  employees  
ORDER BY  
  ReportsTo;
```

	ReportsTo INTEGER
>	(NULL)
>	1
>	1
>	2
>	2
>	2
>	6
>	6

〈그림17_DB_SQL 1_정렬에서의 NULL 예시〉

SELECT statement 실행 순서



<그림18_DB_SQL 1_SELECT statement 실행 순서>

1. 테이블에서 (FROM)
2. 조회하여 (SELECT)
3. 정렬 (ORDER BY)

Filtering data

Filtering data 관련 Keywords

- Clause

- DISTINCT
- WHERE
- LIMIT

- Operator

- BETWEEN
- IN
- LIKE
- Comparison
- Logical



Clause

SQL 문장에서 특정 기능을 수행하도록 지정하는 문장 구성 요소

<개념7_DB_SQL 1_Clause 정의>



Operator

SQL에서 조건을 비교하거나 데이터를 선택하기 위해 사용하는 명령 기호 또는 키워드

<개념8_DB_SQL 1_Operator 정의>

DISTINCT

DISTINCT syntax

```
SELECT DISTINCT  
    select_list  
FROM  
    table_name;
```

- SELECT 키워드 바로 뒤에 작성해야 함
- SELECT DISTINCT 키워드 다음에 고유한 값을 선택하려는 하나 이상의 필드를 지정



DISTINCT statement
조회 결과에서 중복된
레코드를 제거

<개념9_DB_SQL 1_DISTINCT 문 정의>

DISTINCT 활용 전

- 테이블 customers에서 Country 필드의 모든 데이터를 오름차순 조회

Country
NVARCHAR(40)
> Argentina
> Australia
> Austria
> Belgium
> Brazil
> Brazil
> Brazil
> Brazil
> Brazil
> Brazil
> Canada
> Canada
> Canada
> Canada

<그림19_DB_SQL 1_DISTINCT 활용 1 실행 결과>

DISTINCT 활용 전

- 테이블 customers에서 Country 필드의 모든 데이터를 오름차순 조회

```
SELECT  
  Country  
FROM  
  customers  
ORDER BY  
  Country;
```

Country
NVARCHAR(40)
> Argentina
> Australia
> Austria
> Belgium
> Brazil
> Brazil
> Brazil
> Brazil
> Brazil
> Brazil
> Canada
> Canada
> Canada
> Canada

DISTINCT 활용 1

- 테이블 customers에서 Country 필드의 모든 데이터를 중복없이 오름차순 조회

Q	Country NVARCHAR(40)
>	Argentina
>	Australia
>	Austria
>	Belgium
>	Brazil
>	Canada
>	Chile
>	Czech Republic
>	Denmark
>	Finland
>	France
>	Germany
>	Hungary

<그림20_DB_SQL 1_DISTINCT 활용 2 실행 결과>

DISTINCT 활용 1

- 테이블 customers에서 Country 필드의 모든 데이터를 중복없이 오름차순 조회

```
SELECT DISTINCT  
  Country  
FROM  
  customers  
ORDER BY  
  Country;
```

Country
NVARCHAR(40)
> Argentina
> Australia
> Austria
> Belgium
> Brazil
> Canada
> Chile
> Czech Republic
> Denmark
> Finland
> France
> Germany
> Hungary

<그림20_DB_SQL 1_DISTINCT 활용 2 실행 결과>

WHERE

WHERE syntax

```
SELECT
    select_list
FROM
    table_name
WHERE
    search_condition;
```

- FROM clause 뒤에 위치
- search_condition은 비교연산자 및 논리연산자(AND, OR, NOT 등)를 사용하는 구문이 사용됨



WHERE statement
조회 시 특정 검색 조건을 지정

〈개념10_DB_SQL 1_WHERE 문 정의〉



WHERE 활용 1

- 테이블 customers에서 City 필드 값이 'Prague'인 데이터의 LastName, FirstName, City 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	City NVARCHAR(40)
>	Wichterlová	František	Prague
>	Holý	Helena	Prague

<그림21_DB_SQL 1_WHERE 활용 1 실행 결과>

WHERE 활용 1

- 테이블 customers에서 City 필드 값이 'Prague'인 데이터의 LastName, FirstName, City 조회

```
SELECT
    LastName, FirstName, City
FROM
    customers
WHERE
    City = 'Prague';
```

	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	City NVARCHAR(40)
>	Wichterlová	František	Prague
>	Holý	Helena	Prague

<그림21_DB_SQL 1_WHERE 활용 1 실행 결과>

WHERE 활용 2

- 테이블 customers에서 City 필드 값이 'Prague'가 아닌 데이터의 LastName, FirstName, City 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	City NVARCHAR(40)
>	Gonçalves	Luís	São José dos Campos
>	Köhler	Leonie	Stuttgart
>	Tremblay	François	Montréal
>	Hansen	Bjørn	Oslo
>	Gruber	Astrid	Vienne
>	Peeters	Daan	Brussels
>	Nielsen	Kara	Copenhagen
>	Martins	Eduardo	São Paulo
>	Rocha	Alexandre	São Paulo
>	Almeida	Roberto	Rio de Janeiro
>	Ramos	Fernanda	Brasília
>	Philips	Mark	Edmonton
>	Peterson	Jennifer	Vancouver

WHERE 활용 2

- 테이블 customers에서 City 필드 값이 'Prague'가 아닌 데이터의 LastName, FirstName, City 조회

```
SELECT
    LastName, FirstName, City
FROM
    customers
WHERE
    City != 'Prague';
```

	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	City NVARCHAR(40)
>	Gonçalves	Luís	São José dos Campos
>	Köhler	Leonie	Stuttgart
>	Tremblay	François	Montréal
>	Hansen	Bjørn	Oslo
>	Gruber	Astrid	Vienne
>	Peeters	Daan	Brussels
>	Nielsen	Kara	Copenhagen
>	Martins	Eduardo	São Paulo
>	Rocha	Alexandre	São Paulo
>	Almeida	Roberto	Rio de Janeiro
>	Ramos	Fernanda	Brasília
>	Philips	Mark	Edmonton
>	Peterson	Jennifer	Vancouver

WHERE 활용 3

- 테이블 customers에서 Company 필드 값이 NULL이고
Country 필드 값이 'USA'인 데이터의 LastName, FirstName, Company, Country 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	Company NVARCHAR(80)	Country NVARCHAR(40)
>	Brooks	Michelle	(NULL)	USA
>	Miller	Dan	(NULL)	USA
>	Chase	Kathy	(NULL)	USA
>	Leacock	Heather	(NULL)	USA
>	Gordon	John	(NULL)	USA
>	Ralston	Frank	(NULL)	USA
>	Stevens	Victor	(NULL)	USA
>	Cunningham	Richard	(NULL)	USA
>	Gray	Patrick	(NULL)	USA
>	Barnett	Julia	(NULL)	USA

<그림23_DB_SQL1_WHERE 활용 3 실행 결과>

WHERE 활용 3

- 테이블 customers에서 Company 필드 값이 NULL이고

Country 필드 값이 'USA'인 데이터의 LastName, FirstName, Company, Country 조회

```
SELECT
    LastName, FirstName, Company, Country
FROM
    customers
WHERE
    Company IS NULL
    AND Country = 'USA';
```

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	Company NVARCHAR(80)	Country NVARCHAR(40)
>	Brooks	Michelle	(NULL)	USA
>	Miller	Dan	(NULL)	USA
>	Chase	Kathy	(NULL)	USA
>	Leacock	Heather	(NULL)	USA
>	Gordon	John	(NULL)	USA
>	Ralston	Frank	(NULL)	USA
>	Stevens	Victor	(NULL)	USA
>	Cunningham	Richard	(NULL)	USA
>	Gray	Patrick	(NULL)	USA
>	Barnett	Julia	(NULL)	USA

<그림23_DB_SQL 1_WHERE 활용 3 실행 결과>

WHERE 활용 4

- 테이블 customers에서 Company 필드 값이 NULL이거나 Country 필드 값이 'USA'인 데이터의 LastName, FirstName, Company, Country 조회

Q	• LastName NVARCHAR(20)	• FirstName NVARCHAR(40)	Company NVARCHAR(80)	Country NVARCHAR(40)
>	Köhler	Leonie	(NULL)	Germany
>	Tremblay	François	(NULL)	Canada
>	Hansen	Bjørn	(NULL)	Norway
>	Holý	Helena	(NULL)	Czech Republic
>	Gruber	Astrid	(NULL)	Austria
>	Peeters	Daan	(NULL)	Belgium
>	Nielsen	Kara	(NULL)	Denmark
>	Ramos	Fernanda	(NULL)	Brazil
>	Harris	Frank	Google Inc.	USA
>	Smith	Jack	Microsoft Corporation	USA

<그림24_DB_SQL 1_WHERE 활용 4 실행 결과>

WHERE 활용 4

- 테이블 customers에서 Company 필드 값이 NULL이거나 Country 필드 값이 'USA'인 데이터의 LastName, FirstName, Company, Country 조회

```

SELECT
    LastName, FirstName, Company, Country
FROM
    customers
WHERE
    Company IS NULL
    OR Country = 'USA';
  
```

Q	• LastName NVARCHAR(20)	• FirstName NVARCHAR(40)	Company NVARCHAR(80)	Country NVARCHAR(40)
>	Köhler	Leonie	(NULL)	Germany
>	Tremblay	François	(NULL)	Canada
>	Hansen	Bjørn	(NULL)	Norway
>	Holý	Helena	(NULL)	Czech Republic
>	Gruber	Astrid	(NULL)	Austria
>	Peeters	Daan	(NULL)	Belgium
>	Nielsen	Kara	(NULL)	Denmark
>	Ramos	Fernanda	(NULL)	Brazil
>	Harris	Frank	Google Inc.	USA
>	Smith	Jack	Microsoft Corporation	USA

<그림24_DB_SQL 1_WHERE 활용 4 실행 결과>

WHERE 활용 5

- 테이블 tracks에서 Bytes 필드 값이 10,000 이상 500,000 이하인 데이터의 Name, Bytes 조회

	* Name NVARCHAR(200)	Bytes INTEGER
>	Now Sports	161266
>	A Statistic	211997
>	The Real Problem	387360
>	Oprah	224313
>	Commercial 1	319888

<그림25_DB_SQL 1_WHERE 활용 5 실행 결과>

WHERE 활용 5

- 테이블 tracks에서 Bytes 필드 값이 10,000 이상 500,000 이하인 데이터의 Name, Bytes 조회

```
SELECT
    Name, Bytes
FROM
    tracks
WHERE
    Bytes BETWEEN 100000 AND 500000;

-- WHERE
--     Bytes >= 10000
--     AND Bytes <= 50000;
```

	* Name NVARCHAR(200)	Bytes INTEGER
>	Now Sports	161266
>	A Statistic	211997
>	The Real Problem	387360
>	Oprah	224313
>	Commercial 1	319888

<그림25_DB_SQL 1_WHERE 활용 5 실행 결과>

WHERE 활용 6

- 테이블 tracks에서 Bytes 필드 값이 10,000 이상 500,000 이하인 데이터의 Name, Bytes를 Bytes 기준으로 오름차순 조회

	 * Name  	Bytes 
	NVARCHAR(200)	INTEGER
>	Now Sports	161266
>	A Statistic	211997
>	Oprah	224313
>	Commercial 1	319888
>	The Real Problem	387360

<그림26_DB_SQL 1_WHERE 활용 6 실행 결과>

WHERE 활용 6

- 테이블 tracks에서 Bytes 필드 값이 10,000 이상 500,000 이하인 데이터의 Name, Bytes를 Bytes 기준으로 오름차순 조회

```
SELECT
    Name, Bytes
FROM
    tracks
WHERE
    Bytes BETWEEN 100000 AND 500000
ORDER BY Bytes;
```

	* Name NVARCHAR(200)	Bytes INTEGER
>	Now Sports	161266
>	A Statistic	211997
>	Oprah	224313
>	Commercial 1	319888
>	The Real Problem	387360

<그림26_DB_SQL 1_WHERE 활용 6 실행 결과>

WHERE 활용 7

- 테이블 customers에서 Country 필드 값이 'Canada' 또는 'Germany' 또는 'France'인 데이터의 LastName, FirstName, Country 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	Country NVARCHAR(40)
>	Köhler	Leonie	Germany
>	Tremblay	François	Canada
>	Philips	Mark	Canada
>	Peterson	Jennifer	Canada
>	Brown	Robert	Canada
>	Francis	Edward	Canada
>	Silk	Martha	Canada
>	Mitchell	Aaron	Canada
>	Sullivan	Ellie	Canada
>	Schneider	Hannah	Germany
>	Zimmermann	Fynn	Germany
>	Schröder	Niklas	Germany

<그림27_DB_SQL 1_WHERE 활용 7 실행 결과>

WHERE 활용 7

- 테이블 customers에서 Country 필드 값이 'Canada' 또는 'Germany' 또는 'France'인 데이터의 LastName, FirstName, Country 조회

```
SELECT
    LastName, FirstName, Country
FROM
    customers
WHERE
    Country IN ('Canada', 'Germany', 'France');

-- WHERE
-- Country = 'Canada'
-- OR Country = 'Germany'
-- OR Country = 'France';
```

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	Country NVARCHAR(40)
>	Köhler	Leonie	Germany
>	Tremblay	François	Canada
>	Philips	Mark	Canada
>	Peterson	Jennifer	Canada
>	Brown	Robert	Canada
>	Francis	Edward	Canada
>	Silk	Martha	Canada
>	Mitchell	Aaron	Canada
>	Sullivan	Ellie	Canada
>	Schneider	Hannah	Germany
>	Zimmermann	Fynn	Germany
>	Schröder	Niklas	Germany

〈그림27_DB_SQL 1_WHERE 활용 7 실행 결과〉

WHERE 활용 8

- 테이블 customers에서 Country 필드 값이 'Canada' 또는 'Germany' 또는 'France'가 아닌 데이터의 LastName, FirstName, Country 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)	Country NVARCHAR(40)
>	Gonçalves	Luís	Brazil
>	Hansen	Bjørn	Norway
>	Wichterlová	František	Czech Republic
>	Holý	Helena	Czech Republic
>	Gruber	Astrid	Austria
>	Peeters	Daan	Belgium
>	Nielsen	Kara	Denmark
>	Martins	Eduardo	Brazil
>	Rocha	Alexandre	Brazil
>	Almeida	Roberto	Brazil
>	Ramos	Fernanda	Brazil
>	Harris	Frank	USA
>	Smith	Jack	USA

〈그림28_DB_SQL 1_WHERE 활용 8 실행 결과〉

WHERE 활용 8

- 테이블 customers에서 Country 필드 값이

'Canada' 또는 'Germany' 또는 'France'가 아닌 데이터의 LastName, FirstName, Country 조회

```
SELECT
    LastName, FirstName, Country
FROM
    customers
WHERE
    Country NOT IN ('Canada', 'Germany', 'France');
```

Q	• LastName NVARCHAR(2C)	• FirstName NVARCHAR(4C)	Country NVARCHAR(4C)
>	Gonçalves	Luís	Brazil
>	Hansen	Bjørn	Norway
>	Wichterlová	František	Czech Republic
>	Holý	Helena	Czech Republic
>	Gruber	Astrid	Austria
>	Peeters	Daan	Belgium
>	Nielsen	Kara	Denmark
>	Martins	Eduardo	Brazil
>	Rocha	Alexandre	Brazil
>	Almeida	Roberto	Brazil
>	Ramos	Fernanda	Brazil
>	Harris	Frank	USA
>	Smith	Jack	USA

<그림28_DB_SQL 1_WHERE 활용 8 실행 결과>

WHERE 활용 9

- 테이블 customers에서 LastName 필드 값이 'son'으로 끝나는 데이터의 LastName, FirstName 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)
>	Peterson	Jennifer
>	Johansson	Joakim

<그림29_DB_SQL 1_WHERE 활용 9 실행 결과>

WHERE 활용 9

- 테이블 customers에서 LastName 필드 값이 'son'으로 끝나는 데이터의 LastName, FirstName 조회

```
SELECT
    LastName, FirstName
FROM
    customers
WHERE
    LastName LIKE '%son';
```

	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)
>	Peterson	Jennifer
>	Johansson	Joakim

<그림29_DB_SQL 1_WHERE 활용 9 실행 결과>

WHERE 활용 10

- 테이블 customers에서 FirstName 필드 값이 4자리면서 'a'로 끝나는 데이터의 LastName, FirstName 조회

Q	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)
>	Nielsen	Kara
>	Jones	Emma
>	Srivastava	Puja

<그림30_DB_SQL 1_WHERE 활용 10 실행 결과>

WHERE 활용 10

- 테이블 customers에서 FirstName 필드 값이 4자리면서 'a'로 끝나는 데이터의 LastName, FirstName 조회

```
SELECT
    LastName, FirstName
FROM
    customers
WHERE
    FirstName LIKE '___a';
```

	* LastName NVARCHAR(20)	* FirstName NVARCHAR(40)
>	Nielsen	Kara
>	Jones	Emma
>	Srivastava	Puja

<그림30_DB_SQL 1_WHERE 활용 10 실행 결과>

Operators

Comparison Operators

- 비교 연산자
 - =, >=, <=, !=
 - IS , LIKE, IN
 - BETWEEN...AND

Logical Operators

- 논리 연산자
 - AND(&&)
 - OR(||)
 - NOT(!)



IN Operator

값이 특정 목록 안에 있는지 확인

<개념11_DB_SQL 1_IN Operator 정의>



LIKE Operator

값이 특정 패턴에 일치하는지 확인
(Wildcards와 함께 사용)

<개념12_DB_SQL 1_LIKE Operator 정의>

Wildcard

Wildcard Characters

‘ % ’

0개 이상의 문자열과 일치 하는지 확인

‘ _ ’

단일 문자와 일치하는지 확인

LIMIT

LIMIT syntax

```
SELECT  
    select_list  
FROM  
    table_name  
LIMIT [offset,] row_count;
```

- 하나 또는 두 개의 인자를 사용 (0 또는 양의 정수)
- row_count는 조회하는 최대 레코드 수를 지정

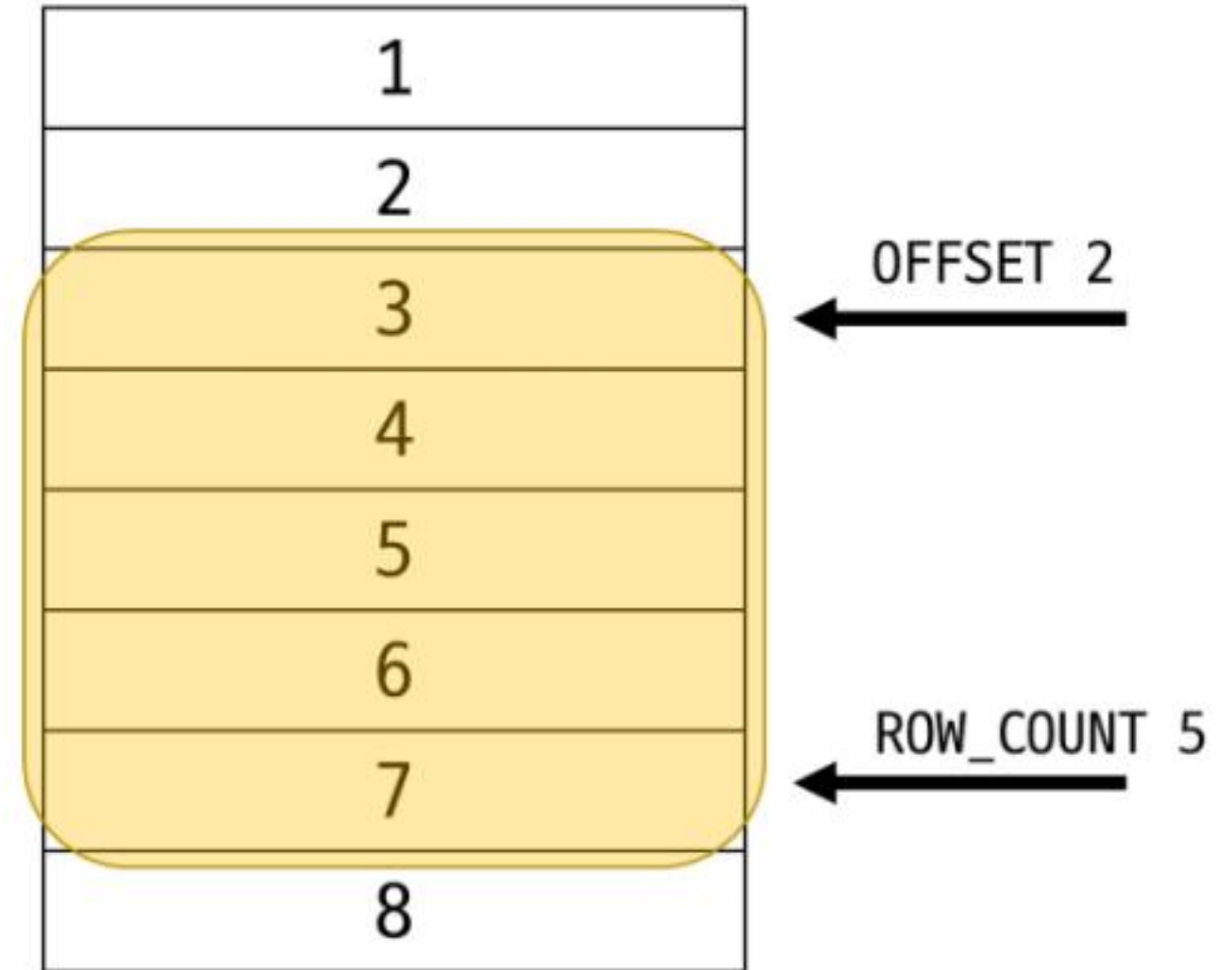


LIMIT statement
조회하는 레코드 수를 제한

〈개념13_DB_SQL 1_LIMIT 문 정의〉

LIMIT & OFFSET 예시

```
SELECT  
  ..  
FROM  
  ..  
LIMIT 2, 5;
```



〈그림31_DB_SQL 1_LIMIT과 OFFSET 예시〉

LIMIT 활용 1

- 테이블 tracks에서 TrackId, Name, Bytes 필드 데이터를 Bytes 기준 내림차순으로 7개만 조회

	TrackId INTEGER	Name NVARCHAR(200)	Bytes INTEGER
>	3224	Through a Looking Glass	1059546140
>	2820	Occupation / Precipice	1054423946
>	3236	The Young Lords	587051735
>	3242	The Man With Nine Lives	577829804
>	2910	Dave	574325829
>	3235	The Magnificent Warriors	570152232
>	3231	The Lost Warrior	558872190

<그림32_DB_SQL1_LIMIT 활용 1 실행 결과>

LIMIT 활용 1

- 테이블 tracks에서 TrackId, Name, Bytes 필드 데이터를 Bytes 기준 내림차순으로 7개만 조회

```
SELECT
    TrackId, Name, Bytes
FROM
    tracks
ORDER BY Bytes DESC
LIMIT 7;
```

	TrackId INTEGER	Name NVARCHAR(200)	Bytes INTEGER
>	3224	Through a Looking Glass	1059546140
>	2820	Occupation / Precipice	1054423946
>	3236	The Young Lords	587051735
>	3242	The Man With Nine Lives	577829804
>	2910	Dave	574325829
>	3235	The Magnificent Warriors	570152232
>	3231	The Lost Warrior	558872190

<그림32_DB_SQL1_LIMIT 활용 1 실행 결과>

LIMIT 활용 2

- 테이블 tracks에서 TrackId, Name, Bytes 필드 데이터를 Bytes 기준 내림차순으로 4번째부터 7번째 데이터만 조회

	TrackId INTEGER	* Name NVARCHAR(200)	Bytes INTEGER
>	3242	The Man With Nine Lives	577829804
>	2910	Dave	574325829
>	3235	The Magnificent Warriors	570152232
>	3231	The Lost Warrior	558872190

<그림33_DB_SQL 1_LIMIT 활용 2 실행 결과>

LIMIT 활용 2

- 테이블 tracks에서 TrackId, Name, Bytes 필드 데이터를
Bytes 기준 내림차순으로 4번째부터 7번째 데이터만 조회

```
SELECT
  TrackId, Name, Bytes
FROM
  tracks
ORDER BY
  Bytes DESC
LIMIT 3, 4;
-- LIMIT 4 OFFSET 3;
```

	TrackId INTEGER	Name NVARCHAR(200)	Bytes INTEGER
>	3242	The Man With Nine Lives	577829804
>	2910	Dave	574325829
>	3235	The Magnificent Warriors	570152232
>	3231	The Lost Warrior	558872190

<그림33_DB_SQL 1_LIMIT 활용 2 실행 결과>

Grouping data

GROUP BY

GROUP BY syntax

```
SELECT  
    c1, c2, ..., cn, aggregate_function(ci)  
FROM  
    table_name  
GROUP BY  
    c1, c2, ..., cn;
```

- FROM 및 WHERE 절 뒤에 배치
- GROUP BY 절 뒤에 그룹화 할 필드 목록을 작성



GROUP BY statement

레코드를 그룹화하여 요약본 생성
(‘집계 함수’와 함께 사용)

<개념14_DB_SQL 1_GROUP BY 문 정의>

집계 함수

Aggregation Functions

집계 함수

값에 대한 계산을 수행하고 단일한 값을 반환하는 함수

SUM, AVG, MAX, MIN, COUNT

GROUP BY 예시 (1/2)

1. Country 필드를 그룹화

```
SELECT
  Country
FROM
  customers
GROUP BY
  Country;
```

Country
NVARCHAR(40)
> Argentina
> Australia
> Austria
> Belgium
> Brazil
> Canada
> Chile
> Czech Republic
> Denmark
> Finland
> France
> Germany
> Hungary

GROUP BY 예시 (2/2)

2. COUNT 함수는 그룹별로 묶은 총 수를 반환

```
SELECT  
    Country, COUNT(*)  
FROM  
    customers  
GROUP BY  
    Country;
```

Q	Country NVARCHAR(40)	COUNT(*)
>	Argentina	1
>	Australia	1
>	Austria	1
>	Belgium	1
>	Brazil	5
>	Canada	8
>	Chile	1
>	Czech Republic	2
>	Denmark	1
>	Finland	1
>	France	5
>	Germany	4
>	Hungary	1

GROUP BY 활용 1

- 테이블 tracks에서 Composer 필드를 그룹화하여 각 그룹에 대한 Bytes의 평균 값을 내림차순 조회

Q	Composer NVARCHAR(220)	AVG(Bytes)
>	(NULL)	97897024.002045
>	Blackmore/Gillan/Glover/Lord/Paice	39267613.0
>	Jimmy Page/Led Zeppelin	36052247.0
>	Gillan/Glover/Lord/Nix - Blackmore/Paice	29846063.0
>	John Bonham/John Paul Jones/Robert Plant/Willie Dixon	28191437.0
>	Blackmore/Coverdale	27775442.0
>	Arthur Crudup/Bernard Besman/Bukka White/Doc Pomus/John Bonham/John Lee Hooker,	26742545.0
>	Blackmore/Coverdale/Lord/Paice	24887209.0
>	Carlos Santana & C. Thompson	24703884.0
>	Chester Burnett/Jimmy Page/John Bonham/John Paul Jones/Robert Plant	23092953.0
>	E.Weiss	22067102.0
>	Whitfield-Strong	21947845.0
>	Diamond/Shermann	21942829.0

<그림36_DB_SQL 1_GROUP BY 활용 1 실행 결과>

GROUP BY 활용 1

- 테이블 tracks에서 Composer 필드를 그룹화하여 각 그룹에 대한 Bytes의 평균 값을 내림차순 조회

```
SELECT
  Composer,
  AVG(Bytes)
FROM
  tracks
GROUP BY
  Composer
ORDER BY
  AVG(Bytes) DESC;
```

Composer NVARCHAR(220)	AVG(Bytes)
> (NULL)	97897024.002045
> Blackmore/Gillan/Glover/Lo	39267613.0
> Jimmy Page/Led Zeppelin	36052247.0
> Gillan/Glover/Lord/Nix - Bla	29846063.0
> John Bonham/John Paul Jor	28191437.0
> Blackmore/Coverdale	27775442.0
> Arthur Crudup/Bernard Besi	26742545.0
> Blackmore/Coverdale/Lord/	24887209.0
> Carlos Santana & C. Thomp	24703884.0
> Chester Burnett/Jimmy Page	23092953.0
> E.Weiss	22067102.0
> Whitfield-Strong	21947845.0
> Diamond/Shermann	21942829.0

GROUP BY 활용 1

- 테이블 tracks에서 Composer 필드를 그룹화하여 각 그룹에 대한 Bytes의 평균 값을 내림차순 조회

```
SELECT
  Composer,
  AVG(Bytes) AS avgOfBytes
FROM
  tracks
GROUP BY
  Composer
ORDER BY
  avgOfBytes DESC;
```

Composer NVARCHAR(220)	avgOfBytes
> (NULL)	97897024.002045
> Blackmore/Gillan/Glover/Lo	39267613.0
> Jimmy Page/Led Zeppelin	36052247.0
> Gillan/Glover/Lord/Nix - Bla	29846063.0
> John Bonham/John Paul Jor	28191437.0
> Blackmore/Coverdale	27775442.0
> Arthur Crudup/Bernard Besi	26742545.0
> Blackmore/Coverdale/Lord/	24887209.0
> Carlos Santana & C. Thomp	24703884.0
> Chester Burnett/Jimmy Pagi	23092953.0
> E.Weiss	22067102.0
> Whitfield-Strong	21947845.0
> Diamond/Shermann	21942829.0

GROUP BY 활용 2

- 테이블 tracks에서 Composer 필드를 그룹화하여
각 그룹에 대한 Milliseconds의 평균 값이 10 미만인 데이터 조회
 - (단, Milliseconds 필드는 60,000으로 나눠 분 단위 값의 평균으로 계산)

```
SELECT
  Composer,
  AVG(Milliseconds / 60000) AS avgOfMinute
FROM
  tracks
WHERE
  avgOfMinute < 10
GROUP BY
  Composer;
```

에러 발생

Invalid use of group function

HAVING

HAVING

HAVING clause

집계 항목에 대한 세부 조건을 지정

주로 GROUP BY와 함께 사용되며 GROUP BY가 없다면 WHERE 처럼 동작

WHERE와 HAVING 비교 (1/2)

WHERE

- 목적
 - 개별 행에 대한 조건을 지정하여 데이터를 필터링
- 적용 시점
 - FROM과 JOIN 등의 단계 이후, GROUP BY 이전에 적용
- 사용 예
 - 특정 조건을 만족하는 행 만을 대상으로 집계나 정렬 등의 작업을 수행할 때 사용

HAVING

- 목적
 - GROUP BY에 의해 그룹화된 결과에 대해 조건을 지정하여 그룹을 필터링
- 적용 시점
 - 그룹핑 및 집계 함수 적용 후에 조건을 평가
- 사용 예
 - 그룹별 집계 결과에 조건을 걸어 특정 그룹만을 선택할 때 사용

WHERE와 HAVING 비교 (2/2)

WHERE

```
SELECT *  
FROM employees  
WHERE salary > 30000;
```

HAVING

```
SELECT department, COUNT(*)  
    AS num_employees  
FROM employees  
GROUP BY department  
HAVING num_employees > 5;
```

SELECT statement 실행 순서



<그림38_DB_SQL 1_SELECT 문 실행 순서 2>

1. 테이블에서 (FROM)
2. 특정 조건에 맞추어 (WHERE)
3. 그룹화 하고 (GROUP BY)
4. 만약 그룹 중에서 조건이 있다면 맞추고 (HAVING)
5. 조회하여 (SELECT)
6. 정렬하고 (ORDER BY)
7. 특정 위치의 값을 가져옴 (LIMIT)

참고

Query

 Query란?

Query

“데이터베이스로부터 정보를 요청” 하는 것
일반적으로 SQL로 작성하는 코드를 쿼리문(SQL문)이라 함

NULL 비교

NULL

- SQL에서 NULL은 실제 값이 아니라 “값이 없음” 또는 “알 수 없음”을 의미
 - 때문에 일반적인 등호(=) 연산자로 NULL을 비교하면 의도한 대로 작동하지 않음

SQL의 3값 논리

- SQL은 논리 연산에 대해 세 가지 값을 사용
 1. TRUE
 2. FALSE
 3. UNKNOWN (알 수 없음)
- 예를 들어, `NULL = NULL`의 결과는 TRUE가 아니라 `UNKNOWN`이 됨
- 이는 두 NULL이 실제로 어떤 값을 가지고 있지 않기 때문

값의 부재와 불확실성

- NULL은 “값이 존재하지 않음”을 나타내므로, 특정 값과 동일하다고 볼 수 없음
- '=' 연산자를 사용하면, NULL은 어떤 값과도 비교할 수 없으므로 결과가 UNKNOWN이 되어 기대한 결과를 얻지 못함

명시적 비교: IS와 IS NOT

- SQL 표준은 NULL 값을 비교할 때 명시적으로 IS NULL 또는 IS NOT NULL 구문을 사용하도록 규정
- WHERE column IS NULL은 해당 컬럼에 값이 없음을 정확하게 확인할 수 있도록 해줌
- 반대로 WHERE column IS NOT NULL은 값이 존재하는 행을 찾음

=와 IS 비교

=

- 일반적인 값의 동등성(같음)을 비교할 때 사용
- 제한 사항
 - 만약 비교하는 값 중 하나라도 NULL이면, 결과는 UNKNOWN이 됨
 - 예를 들어, NULL = NULL의 결과는 TRUE가 아니라 UNKNOWN
 - SQL의 3값 논리(참, 거짓, 알 수 없음) 체계 때문에 이런 결과가 발생

```
SELECT * FROM employees  
WHERE department = 'Sales';
```

IS

- 'NULL'과 같은 특별한 값을 비교할 때 사용
- 사용처
 1. NULL 비교
 2. Boolean 값 비교

```
SELECT * FROM employees  
WHERE department IS NULL;  
...  
WHERE is_active IS TRUE;
```

SQL 표준

SQL 표준

- SQL은 미국 국립 표준 협회(ANSI)와 국제 표준화 기구(ISO)에 의해 표준이 채택됨
- 모든 RDBMS에서 SQL 표준을 지원
- 다만 각 RDBMS마다 독자적인 기능에 따라 표준을 벗어나는 문법이 존재하니 주의

Querying data

- 1888. SQL 조회 - 기초
- 2899. 회원 목록 DB - 조회

Sorting data

- 3050. SQL조회, 정렬
- 2900. 회원 목록 DB - 데이터 정렬과 중복제거

Filtering data

- 2900. 회원 목록 DB - 데이터 정렬과 중복제거
- 2901. 회원 목록 DB - WildCards
- 2903. 회원 목록 DB - 조건부 조회 심화

Grouping data

- 1889. SQL 조회 - 심화
- 3051. SQL 그룹화 조회 - songs
- 2902. 회원 목록 DB - 집계

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

- 1 다음 중 모든 필드를 조회하는 SQL 문은 무엇인가요?

SELECT _____ FROM employees;

- 2 고객 테이블에서 이름과 국가를 조회하려면 어떤 SELECT 문을 작성해야 하나요?

SELECT _____, _____ FROM customers;

- 3 '국가'가 'USA'인 고객만 조회하는 WHERE 절을 작성하세요.

WHERE _____

- 4 FirstName이 'Kim'인 직원의 정보를 조회하려면 어떤 조건문을 사용해야 하나요?

WHERE _____

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

- 5 직원 테이블에서 FirstName만 오름차순 정렬하여 조회하려면 어떻게 하나요?

ORDER BY FirstName _____

- 6 테이블 tracks에서 Milliseconds가 60,000보다 큰 데이터만 조회하려면?

WHERE _____

- 7 중복을 제거하고 모든 국가를 조회하려면 어떤 키워드를 사용해야 하나요?

SELECT _____ country FROM customers;

- 8 고객 테이블에서 이름이 'son'으로 끝나는 사람을 찾는 SQL 조건은?

WHERE LastName _____ '%son'

Q 각 문제에 올바른 답안을 생각해봅시다

9 데이터베이스가 필요한 이유로 가장 적절한 것은?

- a) 컴퓨터에 프로그램을 설치하기 위해
- b) 정보를 문서로 출력하기 위해
- c) 데이터를 체계적으로 저장하고 활용하기 위해
- d) 파일을 삭제하기 쉽게 만들기 위해

10 관계형 데이터베이스에서 데이터를 저장하는 기본 단위는?

- a) 키워드
- b) 테이블
- c) 메모리
- d) 셀

11 기본 키(Primary Key)의 주요 목적은?

- a) 테이블을 정렬하기 위해
- b) 데이터를 백업하기 위해
- c) 행을 고유하게 식별하기 위해
- d) 열의 개수를 제한하기 위해

12 외래 키(Foreign Key)는 언제 사용되는가?

- a) 열의 데이터 형식을 정의할 때
- b) 두 테이블을 연결할 때
- c) 데이터를 삭제할 때
- d) 테이블을 암호화할 때

Q 각 문제에 올바른 답안을 생각해봅시다

13 다음 중 RDBMS에 대한 설명으로 옳은 것은?

- a) 관계 없이 데이터를 자유롭게 저장한다
- b) 정해진 구조 없이 문서를 보관한다
- c) 테이블 간 논리적인 관계를 지원한다
- d) 하드웨어 드라이버를 제어하는 시스템이다

14 SQL은 어떤 목적으로 사용되는 언어인가?

- a) 컴퓨터 그래픽 구현
- b) 데이터베이스 제어와 질의
- c) 운영체제 개발
- d) 하드웨어 명령어 해석

15 SELECT 문은 주로 어떤 작업을 수행하는가?

- a) 데이터를 조회한다
- b) 테이블을 삭제한다
- c) 새로운 테이블을 생성한다
- d) 사용자 권한을 설정한다

16 WHERE 절의 역할로 알맞은 것은?

- a) 결과를 정렬한다
- b) 테이블 이름을 정의한다
- c) 열의 개수를 세어 출력한다
- d) 특정 조건의 데이터를 필터링한다

확인 문제



정답 및 해설

- 1) SELECT * FROM employees; 2) SELECT Name, Country FROM customers; 3) WHERE Country = 'USA'
4) WHERE FirstName = 'Kim' 5) ORDER BY FirstName ASC 6) WHERE Milliseconds > 60000;
7) SELECT DISTINCT country FROM customers; 8) WHERE LastName LIKE '%son'

1. *는 테이블의 모든 필드를 의미합니다. 모든 열을 조회할 때 사용합니다.
2. 조회하려는 컬럼 이름을 쉼표로 구분하여 SELECT 다음에 나열합니다.
3. WHERE절은 조건을 지정하는 구문으로, Country가 'USA'인 행만 조회합니다.
4. FirstName 필드에서 값이 'Kim'인 데이터를 조회하는 조건문입니다.
5. ORDER BY는 정렬을 위한 구문이며, ASC는 오름차순을 의미합니다.

6. Milliseconds 필드의 값이 60000보다 큰 레코드만 조회합니다.
7. DISTINCT는 중복을 제거하고 고유한 값만 조회할 때 사용합니다.
8. LIKE 연산자와 % 와일드카드를 사용하면 특정 패턴과 일치하는 값을 조회할 수 있습니다.

확인 문제



정답 및 해설

9) c) 데이터를 체계적으로 저장하고 활용하기 위해 10) b) 테이블 11) c) 행을 고유하게 식별하기 위해 12) b) 두 테이블을 연결할 때 13) c) 테이블 간 논리적인 관계를 지원한다 14) b) 데이터베이스 제어와 질의 15) a) 데이터를 조회한다 16) d) 특정 조건의 데이터를 필터링한다

9. 데이터베이스는 정보를 구조적으로 저장하고 검색, 수정, 삭제할 수 있게 해주는 시스템입니다.
10. 관계형 데이터베이스는 데이터를 행과 열로 구성된 테이블 단위로 저장합니다.
11. 테이블 내 각 행(레코드)을 유일하게 구별하기 위한 값이 기본 키입니다.
12. 외래 키는 다른 테이블의 기본 키를 참조하여 두 테이블 간 관계를 설정하는 데 사용됩니다.
13. RDBMS는 관계형 데이터베이스를 관리하는 시스템으로, 테이블 간 관계를 설정할 수 있습니다.

14. SQL은 데이터를 조회하고 조작하는 질의 언어로, 데이터 베이스와의 상호작용에 사용됩니다.
15. WHERE 절은 지정한 조건에 맞는 데이터만 추출할 수 있도록 필터링합니다.
16. WHERE 절은 지정한 조건에 맞는 데이터만 추출할 수 있도록 필터링합니다.

활동 정리

핵심 키워드

개념	설명	예시
데이터 (Data)	저장이나 처리를 위해 구조화된 정보	"서울", 25, True
데이터베이스 (DB)	데이터를 체계적으로 저장하고 관리하는 시스템	고객 정보, 주문 내역 등을 저장하는 구조화된 저장소
관계형 데이터베이스	관련 있는 데이터를 테이블 형태로 저장하고 연결하는 데이터베이스	고객 테이블과 주문 테이블을 고객 ID로 연결
테이블 (Table)	데이터를 행과 열의 구조로 저장하는 단위	employees, orders
행 (Row)	테이블의 한 줄, 하나의 개별 데이터 묶음	고객 한 명의 정보 (이름, 주소, 전화번호 등)
열 (Column)	테이블의 세로 방향 항목, 속성(attribute)	이름, 이메일, 전화번호
기본 키 (Primary Key)	각 행을 고유하게 식별하는 필드	고객번호, 주민등록번호 등
외래 키 (Foreign Key)	다른 테이블의 기본 키를 참조하는 필드	주문 테이블의 고객 ID 필드

<표5_DB_SQL 1_핵심 키워드 1>

활동 정리

핵심 키워드

개념	설명	예시
SQL	관계형 데이터베이스를 조작하기 위한 언어	SELECT * FROM customers;
SELECT	테이블에서 원하는 데이터를 조회하는 SQL 구문	SELECT name FROM employees;
WHERE	조회 조건을 지정하는 절	WHERE age >= 20
ORDER BY	조회 결과를 정렬하는 절	ORDER BY age DESC
DISTINCT	중복을 제거하고 고유한 값만 조회하는 키워드	SELECT DISTINCT country FROM customers ;
LIMIT	조회할 행의 개수를 제한하는 절	LIMIT 10 또는 LIMIT 3, 5
GROUP BY	필드를 기준으로 데이터를 그룹화하는 절	GROUP BY country
HAVING	그룹화된 결과에 조건을 지정하는 절	HAVING AVG(sales) > 100000
연산자	값을 비교하거나 논리 조건을 만들 때 사용하는 키워드 또는 기호	=, !=, IN, BETWEEN, LIKE, AND, OR, NOT 등

<표6_DB_SQL 1_핵심 키워드 2>

활동 정리

오늘 공부한 내용 요약 및 정리

• 데이터와 데이터베이스

- 데이터는 컴퓨터가 이해할 수 있도록 정리된 정보이다.
- 데이터베이스(DB)는 데이터를 체계적으로 저장하고 관리하는 시스템이다.
- 파일이나 스프레드시트 기반 저장 방식은 확장성과 정확성에 한계가 있다.

• 관계형 데이터베이스 (RDB)

- 데이터를 테이블(표) 단위로 저장하고, 열(Column)과 행(Row)으로 구성된다.
- 테이블 간 관계를 설정해 중복을 줄이고 효율적인 데이터 조회가 가능하다.
- 기본 키(PK)는 각 행을 고유하게 식별하고, 외래 키(FK)는 테이블 간 연결에 사용된다.

• DBMS와 RDBMS

- DBMS는 데이터를 저장, 수정, 조회, 삭제할 수 있게 해주는 소프트웨어 시스템이다.
- RDBMS는 관계형 구조를 지원하는 DBMS로, 여러 테이블을 논리적으로 연결할 수 있다

활동 정리

오늘 공부한 내용 요약 및 정리

- SQL (Structured Query Language)
- 주요 문장 구성 요소는 다음과 같다.
 - SELECT: 원하는 데이터를 조회
 - FROM: 데이터를 조회할 테이블 지정
 - WHERE: 조건 설정
 - ORDER BY: 정렬
 - GROUP BY, HAVING: 그룹화 및 그룹 조건 필터링
 - LIMIT: 조회 결과 수 제한
- SQL 실행 흐름
 - FROM - 어떤 테이블에서
 - WHERE - 어떤 조건으로
 - GROUP BY - 어떻게 그룹화하고
 - HAVING - 어떤 그룹을 걸러낼지
 - SELECT - 어떤 데이터를 조회할지
 - ORDER BY - 어떤 순서로 정렬할지
 - LIMIT - 몇 개만 보여줄지

활동 정리

SQL 활용 방법

- "고객 정보는 어떻게 저장되고 있을까?"

1. 수많은 고객의 이름, 주소, 주문 내역을 모두 엑셀로 관리할 수 있을까요?
2. 같은 이름을 가진 고객이 여러 명이라면 어떻게 구별해야 할까요?
3. 고객이 한 명이라도 주문을 여러 번 했다면, 그 관계는 어떻게 저장되어야 할까요?

- 데이터를 구조적으로 저장하는 관계형 데이터베이스의 방식을 이해할 수 있어요.
- 고객 정보에서 원하는 조건의 데이터만 SQL로 정확하게 조회하는 방법을 배울 수 있어요.
- 테이블 간 관계를 맺는 방식과 이를 활용하는 기본 쿼리 작성법을 익힐 수 있어요

감사합니다

